

Lab 4: Persistence, Autoscaling & Rolling Updates

Closing the Durability Gap, Scaling Under Load, and Shipping Safely

What You Will Do

This lab closes three remaining gaps in the stack built across Labs 1 through 3. First, PostgreSQL is given persistent storage that survives Pod restarts -- the gap deliberately left open in Lab 1. Second, the API is connected to a Horizontal Pod Autoscaler and put under real load to watch it scale up and back down. Third, a new version of the API is rolled out with zero downtime, then deliberately rolled back, to prove deployments are safe to ship and safe to undo.

Learning Objectives

- Understand PersistentVolumes and PersistentVolumeClaims, and prove data survives a Pod deletion
- Install and patch metrics-server, the component HPA depends on for real usage data
- Create a HorizontalPodAutoscaler and observe a full scale-up and scale-down cycle under generated load
- Understand why resource requests are mandatory for HPA to function at all
- Perform a rolling update and a rollback, and read Deployment revision history

Prerequisites

- Labs 1-3 completed: postgres, api, and frontend Pods all Running
- Ingress working: `http://localhost/` loads the Angular app and successfully calls `/api/db-check`

Part A — Persistent Storage for PostgreSQL

In Lab 1, deleting a Pod proved Kubernetes' self-healing behavior -- but it also proved something uncomfortable: the replacement Pod got a completely empty filesystem. Any data PostgreSQL had written was gone. This part closes that gap before moving on to anything else, since a production database without durable storage is not a usable database.

Step A1 — Understand the two objects involved

PersistentVolume (PV) vs PersistentVolumeClaim (PVC)

A PersistentVolume is a piece of real storage in the cluster -- a chunk of disk space, independent of any particular Pod's lifecycle.

A PersistentVolumeClaim is a request for storage made by a Pod -- "I need 1Gi of space." Kubernetes matches claims to available volumes.

This separation means storage survives even if the Pod requesting it is deleted and recreated. The new Pod's claim binds to the same existing volume, not a new, empty one.

Step A2 — Create the PersistentVolumeClaim

On Docker Desktop, a default StorageClass automatically provisions the underlying volume for you, so only a PVC needs to be defined explicitly.

04-postgres-pvc.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

ReadWriteOnce means the volume can be mounted as read-write by a single node at a time -- the correct mode for a single-instance database like this one.

Step A3 — Update the PostgreSQL Deployment to use it

Two additions to the existing Deployment from Lab 2: a volumeMounts entry inside the container, and a volumes entry at the Pod spec level referencing the PVC.

04-postgres-with-storage.yaml

```
apiVersion: apps/v1
kind: Deployment
```

```

metadata:
  name: postgres
  labels:
    app: postgres
spec:
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:16
          ports:
            - containerPort: 5432
          env:
            - name: POSTGRES_DB
              value: appdb
            - name: POSTGRES_USER
              value: appuser
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: postgres-secret
                  key: DB_PASSWORD
            - name: PGDATA
              value: /var/lib/postgresql/data/pgdata
          volumeMounts:
            - name: postgres-storage
              mountPath: /var/lib/postgresql/data
      volumes:
        - name: postgres-storage
          persistentVolumeClaim:
            claimName: postgres-pvc
---
apiVersion: v1
kind: Service
metadata:
  name: postgres
spec:
  selector:
    app: postgres
  ports:
    - port: 5432
      targetPort: 5432

```

Why the PGDATA environment variable is added

Postgres' own data directory and the volume mount point both default to `/var/lib/postgresql/data`. Mounting a fresh volume directly there can conflict with a hidden lost+found directory some storage drivers create. Pointing PGDATA at a subfolder (pgdata) inside the mount avoids that conflict entirely.

Step A4 — Apply the updated storage configuration

```
kubectl apply -f 04-postgres-pvc.yaml
kubectl apply -f 04-postgres-with-storage.yaml
kubectl get pvc
kubectl get pods --watch
```

Confirm the PVC shows STATUS Bound, and the postgres pod restarts and reaches 1/1 Running again.

```
kubectl get pvc
persistentvolumeclaim/postgres-pvc unchanged
deployment.apps/postgres unchanged
service/postgres unchanged
NAME          STATUS    VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
postgres-pvc  Bound    pvc-b9f93ec9-6239-466c-8283-55fbd47594c4  1Gi        RWX            standard      <unset>                 3d17h
laxmanp@W4N00TW13C-laxmanp manifests %
```

Step A5 — Prove data survives a Pod deletion

1. Open a shell into the postgres pod and connect with psql.
2. Create a table and insert one row.
3. Exit, then delete the postgres pod directly.
4. Wait for the replacement pod to reach Running.
5. Connect again and confirm the table and row still exist.

```
kubectl exec -it postgres-848f49ccd-4dljt -- psql -U appuser -d appdb

-- inside psql:
CREATE TABLE proof (id SERIAL PRIMARY KEY, note TEXT);
INSERT INTO proof (note) VALUES ('data survived a pod restart');
\q

kubectl delete pod postgres-848f49ccd-4dljt
kubectl get pods --watch
```

Once the new pod is Running, reconnect and check:

```
kubectl exec -it postgres-848f49ccd-bg2nr -- psql -U appuser -d appdb -c "SELECT *
FROM proof;"
```

The row should still be there. Compare this directly to Lab 1, where deleting the unmounted Postgres pod lost everything -- the only difference now is the PersistentVolumeClaim.

```
laxmanp@w4N00Tw13C-laxmanp manifests % kubectl exec -it postgres-848f49ccd-4dljt -- psql -U appuser -d appdb
INSERT 0 1
appdb=# \q
laxmanp@w4N00Tw13C-laxmanp manifests % kubectl delete pod postgres-848f49ccd-4dljt
pod "postgres-848f49ccd-4dljt" deleted from default namespace
laxmanp@w4N00Tw13C-laxmanp manifests % kubectl get pods --watch
NAME                                READY    STATUS    RESTARTS   AGE
api-65c4f494df-cvv75                1/1      Running   1 (33m ago)  3d19h
frontend-6695bbf6b8-knrzz           1/1      Running   1 (33m ago)  3d18h
frontend-6695bbf6b8-w57qg           1/1      Running   1 (33m ago)  3d18h
postgres-848f49ccd-bg2nr             1/1      Running   0           26s
laxmanp@w4N00Tw13C-laxmanp manifests % kubectl exec -it postgres-848f49ccd-bg2nr -- psql -U appuser -d appdb -c "SELECT * FROM proof;"
id |
----+-----
 1 | data survived a pod restart
 2 | data survived a pod restart
 3 | data survived a pod restart
(3 rows)
laxmanp@w4N00Tw13C-laxmanp manifests %
```

Part B — Autoscaling Under Load

Step B1 — Why resource requests are mandatory for this to work at all

The HorizontalPodAutoscaler's entire logic is a percentage: "this pod is using X% of what it requested." Without a CPU request defined on the container, there is no denominator for that percentage, and HPA cannot make a scaling decision. This is the single most common reason HPA appears not to work in real clusters -- the request was simply never set. The api Deployment from Lab 2 already defines:

```
resources:
  requests:
    cpu: "100m"
  limits:
    cpu: "500m"
```

This is what makes everything in this section possible.

Step B2 — Install metrics-server

HPA does not measure CPU or memory itself -- it asks a separate component, metrics-server, for that data. Docker Desktop does not install this by default.

```
kubectl apply -f
https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components
.yaml

kubectl get pods -n kube-system
```

Look for a pod named metrics-server-... Note its READY column -- 0/1 at this stage is expected and addressed in the next step.

Step B3 — Patch metrics-server for local clusters

Why this step is needed

metrics-server tries to verify the node's TLS certificate strictly by default. Local clusters with self-signed certificates fail this check, which shows up as the pod staying at 0/1 Ready, and a log entry similar to: x509: cannot validate certificate ... because it doesn't contain any IP SANs. This is a routine, expected step for any local kind or Docker Desktop cluster -- not something to do on a real production cluster with properly issued certificates.

Confirm the cause first:

```
kubectl logs -n kube-system deployment/metrics-server
```

Then patch the deployment to skip strict TLS verification. On Windows cmd.exe, the quoting must use escaped double quotes, not single quotes, around the JSON patch:

```
kubectl patch deployment metrics-server -n kube-system --type=json
-p='[{"op": "add", "path": "/spec/template/spec/containers/0/args/-", "value": "--kubelet-insecure-tls"}]'
```

If the quoting still causes errors in your shell, use the interactive editor instead, which avoids the issue entirely:

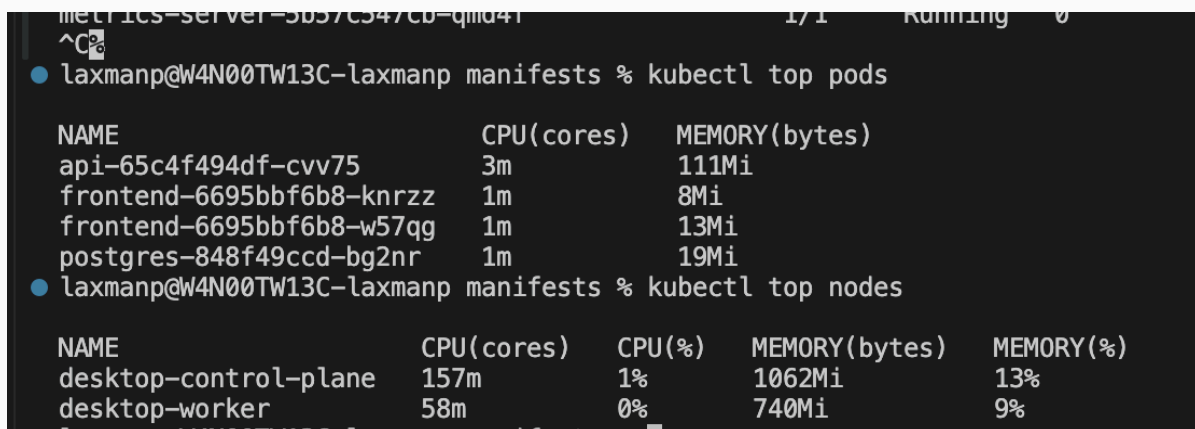
```
kubectl edit deployment metrics-server -n kube-system
# add this line under the existing args: list, then save and close:
# - --kubelet-insecure-tls
```

```
kubectl get pods -n kube-system --watch
```

Wait for the new metrics-server pod to reach 1/1 Running, then confirm it is genuinely serving data:

```
kubectl top pods
kubectl top nodes
```

Real CPU and memory numbers, not an error, confirm metrics-server is fully functional and HPA now has something to read.



```
metrics-server-5b57c547cb-qm4d4 1/1 Running 0
^C
● laxmanp@W4N00TW13C-laxmanp manifests % kubectl top pods

NAME                                CPU(cores)   MEMORY(bytes)
api-65c4f494df-cvv75                3m           111Mi
frontend-6695bbf6b8-knrzz           1m           8Mi
frontend-6695bbf6b8-w57qg           1m           13Mi
postgres-848f49ccd-bg2nr            1m           19Mi
● laxmanp@W4N00TW13C-laxmanp manifests % kubectl top nodes

NAME                    CPU(cores)   CPU(%)   MEMORY(bytes)   MEMORY(%)
desktop-control-plane    157m         1%       1062Mi          13%
desktop-worker            58m         0%       740Mi           9%
```

S

Step B4 — Create the HorizontalPodAutoscaler

04-api-hpa.yaml

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api
  minReplicas: 1
  maxReplicas: 5
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50
```

scaleTargetRef points this HPA at the api Deployment specifically. minReplicas and maxReplicas set the floor and ceiling it will never cross. averageUtilization: 50 means: keep average CPU usage across all pods at roughly 50% of what each pod requested (50% of the 100m request set in Lab 2 is 50m) -- sustained movement past that threshold triggers scaling in either direction.

```
kubectl apply -f 04-api-hpa.yaml
kubectl get hpa
```

At idle, expect a reading close to 1%/50%, with REPLICAS at 1 -- no load yet, so nothing to react to.

The terminal screenshot shows a user at a desktop-worker machine with 58m memory, 0% usage, and 740Mi free. The user runs 'kubectl apply -f 04-api-hpa.yaml' and 'kubectl get hpa'. The output shows the HPA 'api-hpa' is created, targeting the 'api' deployment with a CPU utilization target of 50%. The current state shows 1 replica, 5 max pods, and 0% utilization.

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
api-hpa	Deployment/api	cpu: <unknown>/50%	1	5	0	0s

Step B5 — Generate real load and watch it scale up

Run a disposable pod inside the cluster that hammers the API continuously:

```
kubectl run load-generator --image=busybox --restart=Never -- /bin/sh -c "while true;
do wget -q -O- http://api:8080/ ; done"
```

In a separate terminal, watch the HPA react live:


```
kubectl get hpa --watch
```

Give it one to two minutes -- HPA polls periodically and wants sustained load, not a single spike, before acting. Watch TARGETS climb past 50% and REPLICAS increase as a result, up to the maxReplicas ceiling of 5.

```
laxmanp@W4N00TW13C-laxmanp DOCKER % kubectl get hpa --watch
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
api-hpa	Deployment/api	cpu: 1%/50%	1	5	1	82s
api-hpa	Deployment/api	cpu: 2%/50%	1	5	1	90s
api-hpa	Deployment/api	cpu: 46%/50%	1	5	1	105s
api-hpa	Deployment/api	cpu: 290%/50%	1	5	1	2m
api-hpa	Deployment/api	cpu: 228%/50%	1	5	4	2m15s
api-hpa	Deployment/api	cpu: 118%/50%	1	5	5	2m30s
api-hpa	Deployment/api	cpu: 108%/50%	1	5	5	2m45s

Step B6 — Remove the load and watch it scale back down

```
kubectl delete pod load-generator
kubectl get hpa --watch
```

Why scaling down takes longer than expected

HPA applies a scale-down stabilization window (5 minutes by default), and within that window it does not simply wait -- it keeps a rolling history of recent recommendations and always applies the highest one seen in that window, not the most current reading. Run `kubectl describe hpa api-hpa` and check the Conditions section: a `ScaleDownStabilized` reason with a message like "recent recommendations were higher than current one" confirms this is expected behavior, timed from the most recent scale-up event, not from when load actually stopped.

Keep watching for a few minutes. REPLICAS should step back down toward 1 once the stabilization window from the last scale-up event has fully elapsed.

```
kubectl describe hpa api-hpa
```

Check the Events section at the bottom for `SuccessfulRescale` entries, and the Conditions section for `AbleToScale` and its current Reason -- this is the authoritative source for what HPA is

doing and why, at any moment.

```

pod "load-generator" deleted from default namespace
laxmanp@W4N00TW13C-laxmanp manifests % kubectl describe hpa api-hpa
Name:                                api-hpa
Namespace:                          default
Labels:                             <none>
Annotations:                         <none>
CreationTimestamp:                  Mon, 29 Jun 2026 10:51:30 +0530
Reference:                          Deployment/api
Metrics:
  resource cpu on pods (as a percentage of request): 1% (1m) / 50%
Min replicas:                        1
Max replicas:                        5
Deployment pods:                     5 current / 5 desired
Conditions:
  Type            Status  Reason                        Message
  ----            -
AbleToScale      True    ScaleDownStabilized          recent recommendations were higher than current one, applying the highest recent
commendation
ScalingActive    True    ValidMetricFound             the HPA was able to successfully calculate a replica count from cpu resource uti
ation (percentage of request)
ScalingLimited   True    TooManyReplicas              the desired replica count is more than the maximum replica count
Events:
  Type            Reason                  Age    From                      Message
  ----            -
Normal           SuccessfulRescale       2m57s  horizontal-pod-autoscaler  New size: 4; reason: cpu resource utilization (percentage of request) above target
Normal           SuccessfulRescale       2m42s  horizontal-pod-autoscaler  New size: 5; reason: cpu resource utilization (percentage of request) above target
laxmanp@W4N00TW13C-laxmanp manifests %

```

Part C — Rolling Updates & Rollback

A Deployment manages not just how many replicas exist, but which version they are running, gradually swapping old pods for new ones rather than killing everything at once. This part ships a small, visible change to the API and proves it can be shipped safely, and undone safely.

Step C1 — Make a visible change

In Program.cs, change the version reported by the root endpoint:

```
// Before:
return Results.Ok(new { message = "API is running", version = "1.0" });

// After:
return Results.Ok(new { message = "API is running", version = "2.0" });
```

Step C2 — Build and load the new version under a distinct tag

Always tag versions distinctly, never reuse :latest for this

Using a separate, meaningful tag per build (v2 here, not latest) is what makes rollback possible later. If every build shares the same tag, Kubernetes has no way to distinguish which version a given revision actually ran, and there is nothing concrete to roll back to.

```
docker build -t myapi:v2 .
docker save myapi:v2 | docker exec -i desktop-control-plane ctr -n=k8s.io images
import -
```

(Substitute your actual kind node container name if different from desktop-control-plane.)

Step C3 — Trigger the rolling update

```
kubectl set image deployment/api api=myapi:v2
kubectl rollout status deployment/api
kubectl get pods --watch
```

Watch closely: a new pod running myapi:v2 starts, becomes Ready (passing its readinessProbe), and only then does the old pod terminate. This overlap, however brief, is the entire point of a rolling update -- the old version keeps serving traffic until the new version has

proven itself ready, so there is no window where zero pods are available.

```

importing... elapsed: 1.7 s total: 8.0 s (8.0 B/s)
○ laxmanp@W4N00TW13C-laxmanp MyApi % kubectl set image deployment/api api=myapi:v2
kubectl rollout status deployment/api
kubectl get pods --watch

deployment.apps/api image updated
Waiting for deployment "api" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "api" rollout to finish: 1 old replicas are pending termination...
deployment "api" successfully rolled out

```

NAME	READY	STATUS	RESTARTS	AGE
api-65c4f494df-cvv75	1/1	Terminating	1 (92m ago)	3d20h
api-79cbdb97fb-jvp5p	1/1	Running	0	7s
frontend-6695bbf6b8-knrzz	1/1	Running	1 (92m ago)	3d19h
frontend-6695bbf6b8-w57qg	1/1	Running	1 (92m ago)	3d19h
postgres-848f49ccd-bg2nr	1/1	Running	0	59m
api-65c4f494df-cvv75	0/1	Completed	1 (92m ago)	3d20h
api-65c4f494df-cvv75	0/1	Completed	1 (92m ago)	3d20h
api-65c4f494df-cvv75	0/1	Completed	1 (92m ago)	3d20h
postgres-848f49ccd-bg2nr	1/1	Running	0	59m
frontend-6695bbf6b8-knrzz	1/1	Running	1 (93m ago)	3d19h

Step C4 — Confirm via revision history

```
kubectl rollout history deployment/api
```

Why CHANGE-CAUSE often shows <none>

Kubernetes always tracks revisions automatically and reliably -- this column is purely cosmetic. It only gets populated if a change-cause annotation or the (now deprecated in newer kubectl versions) --record flag was used at the time of the change. An empty CHANGE-CAUSE does not mean history was lost; it means the human-readable reason was never recorded. The per-revision template is still available in full regardless.

To see exactly what changed in any revision, view its full pod template directly:

```
kubectl rollout history deployment/api --revision=2
```

Confirm the most recent revision shows Image: myapi:v2.

```
laxmanp@W4N00TW13C-laxmanp MyApi % kubectl rollout history deployment/api

deployment.apps/api
REVISION  CHANGE-CAUSE
1          <none>
2          <none>

laxmanp@W4N00TW13C-laxmanp MyApi % kubectl rollout history deployment/api --revision=2
deployment.apps/api with revision #2
Pod Template:
  Labels:    app=api
            pod-template-hash=79cddb97fb
  Containers:
    api:
      Image:    myapi:v2
      Port:     8080/TCP (http)
      Host Port: 0/TCP (http)
      Limits:
        cpu:    500m
        memory: 512Mi
      Requests:
        cpu:    100m
        memory: 128Mi
      Liveness:  http-get http://:8080/ delay=10s timeout=5s period=10s #success=1 #failure=3
      Readiness: http-get http://:8080/ delay=5s timeout=3s period=5s #success=1 #failure=2
      Environment Variables from:
        app-config      ConfigMap      Optional: false
      Environment:
        DB_PASSWORD:    <set to the key 'DB_PASSWORD' in secret 'postgres-secret'>      Optional: false
      Mounts:           <none>
      Volumes:          <none>
      Node-Selectors:   <none>
      Tolerations:     <none>
```

Step C5 — Roll back

Suppose v2 turned out to be broken. Reverting requires no YAML editing at all:

```
kubectl rollout undo deployment/api
kubectl rollout status deployment/api
```

Confirm the image reverted to the previous version:

```
kubectl get deployment api -o jsonpath="{.spec.template.spec.containers[0].image}"
```

This single command reverted a live deployment to its previous known-good state. The same rolling mechanism applies in reverse -- the old (now "new again") version is brought up and proven Ready before the broken version is removed.

```
laxmanp@W4N00TW13C-laxmanp MyApi % kubectl rollout undo deployment/api
deployment.apps/api rolled back
laxmanp@W4N00TW13C-laxmanp MyApi % kubectl rollout status deployment/api
Waiting for deployment "api" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "api" rollout to finish: 1 old replicas are pending termination...
deployment "api" successfully rolled out
laxmanp@W4N00TW13C-laxmanp MyApi % kubectl get deployment api -o jsonpath="{.spec.template.spec.containers[0].image}"
myapi:latest
laxmanp@W4N00TW13C-laxmanp MyApi %
```

Troubleshooting Reference

Symptom	Likely Cause / Fix
PVC stuck in Pending	No StorageClass available to satisfy the claim. Confirm with kubectl get storageclass that a default class exists.

Postgres pod fails to start after adding the volume	Check kubectl logs <pod-name> for a permissions or lost+found error; confirm PGDATA points to a subfolder, not the mount root.
kubectl top pods returns an error, not numbers	metrics-server is not Ready yet, or the --kubelet-insecure-tls patch was not applied or has not taken effect. Re-check kubectl get pods -n kube-system.
HPA TARGETS shows <unknown>/50%	Same root cause as above -- HPA cannot read metrics-server data yet. Resolve that first; HPA itself is not broken.
HPA never scales up despite load-generator running	Confirm the api Deployment actually defines resources.requests.cpu. Without it, HPA has no baseline percentage to calculate against.
Replicas stay high long after load-generator is deleted	Expected. Check kubectl describe hpa api-hpa for a ScaleDownStabilized condition -- the 5-minute window is timed from the last scale-up event, and applies the highest recent recommendation, not the current reading.
kubectl patch with --type=json fails on Windows with a quoting error	cmd.exe does not strip single quotes the way bash does. Use escaped double quotes as shown in Step B3, or use kubectl edit instead.
New pod never becomes Ready during a rollout, old pod never terminates	The new image is likely failing its readinessProbe. Check kubectl logs on the new pod and kubectl describe pod for probe failure details.

Key takeaway

Every mechanism in this lab depends on the same underlying idea from Lab 1: Kubernetes continuously reconciles declared state against actual state. A PersistentVolumeClaim declares storage should persist regardless of which Pod is attached to it. An HPA declares a target utilization and lets the control loop add or remove replicas to match it. A rolling update declares a new Pod template and lets the control loop replace old Pods with new ones, never all at once. Nothing here is a new mechanism -- it is the same reconciliation loop from Hour 1, applied to storage, to scale, and to version.